

Badji-Mokhtar Annaba University

Department of Engineer Computer Science

Practical Work N°4

Engineering and Securing Machine Learning Models in
Cybersecurity

Phishing Detection and Adversarial Attacks

Prepared by:
Salah Hacen Nasrallah Zaoui

April 22, 2026

Contents

1	Introduction	3
1.1	Context	3
1.2	TP Objectives	3
1.3	Report Structure	3
2	Data Collection and Feature Extraction	4
2.1	Data Collection	4
2.1.1	Phishing URLs	4
2.1.2	Benign URLs	4
2.2	Dataset Balancing	4
2.3	Feature Extraction	4
2.3.1	Structural Features	5
2.3.2	Keyword Features	5
2.3.3	Security Features	5
2.3.4	URL Shorteners List	5
2.3.5	Brand Domains List	5
3	Model Building and Evaluation	6
3.1	Classifier Choice	6
3.2	Model Parameters	6
3.3	10-Fold Cross Validation	6
3.4	Cross Validation Results	7
3.5	Confusion Matrix	7
3.6	Performance Metrics	7
3.7	Feature Importance	7
3.8	Model Deployment	8
4	Adversarial Attacks	8
4.1	Attack 1: Data Poisoning (Label Flipping)	8
4.1.1	Principle	8
4.1.2	Implementation	8
4.1.3	Results	9
4.1.4	Conclusion	9
4.2	Attack 2: Backdoor/Trigger Injection	9
4.2.1	Principle	9
4.2.2	Trigger Design	9
4.2.3	Implementation	10
4.2.4	Results	10
4.2.5	Conclusion	10
4.3	Attack 3: Evasion Attack	10
4.3.1	Principle	10
4.3.2	Strategies Tested	11
4.3.3	Analysis of Results	11
4.3.4	Conclusion	11

5	Discussion and Proposed Defenses	12
5.1	Summary of Attacks	12
5.2	Proposed Defenses	12
5.2.1	Against Data Poisoning	12
5.2.2	Against Backdoor Injection	12
5.2.3	Against Evasion Attacks	12
5.3	Model Strengths and Weaknesses	13
6	Conclusion	14
6.1	Summary of Achievements	14
6.2	Key Takeaways	14
6.3	Final Remarks	14

1 Introduction

1.1 Context

With the exponential increase in cyberattacks, phishing remains one of the most widespread and dangerous threats. Attackers use sophisticated techniques to trick users and steal their sensitive information. In this context, Machine Learning has become an essential tool for detecting these attacks.

1.2 TP Objectives

This practical work aims to:

- Build and evaluate a phishing detection model based on URL features
- Understand and implement three types of adversarial attacks:
 - Data Poisoning Attack
 - Backdoor/Trigger Injection Attack
 - Evasion Attack
- Measure the model's robustness against these attacks
- Propose appropriate defenses

1.3 Report Structure

This report is organized as follows:

- **Section 2:** Data Collection and Feature Extraction
- **Section 3:** Model Building and Evaluation
- **Section 4:** Adversarial Attacks
- **Section 5:** Discussion and Defenses
- **Section 6:** Conclusion

2 Data Collection and Feature Extraction

2.1 Data Collection

2.1.1 Phishing URLs

Phishing URLs were collected from PhishTank (<https://www.phishtank.com/>), a collaborative platform for reporting fraudulent sites. After cleaning and removing duplicates, we obtained **57,424 phishing URLs**.

2.1.2 Benign URLs

Legitimate URLs came from two sources:

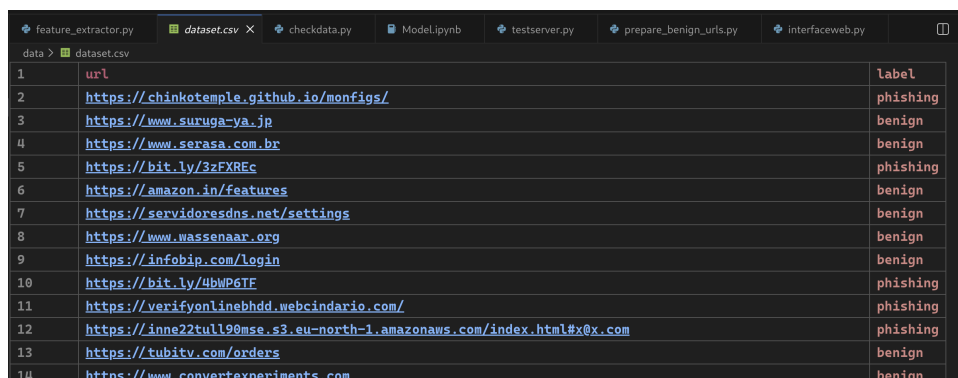
- Tranco List (<https://tranco-list.eu/>): Top 30,000 most visited domains
- Cloudflare Radar: Top 100 domains with categories

After crawling and cleaning, we obtained **57,424 benign URLs**.

2.2 Dataset Balancing

The final dataset is perfectly balanced:

- Phishing URLs: 57,424 (50%)
- Benign URLs: 57,424 (50%)
- Total: 114,848 URLs



	url	label
1	https://chinkotemple.github.io/monfigs/	phishing
2	https://www.suruga-ya.jp	benign
3	https://www.serasa.com.br	benign
4	https://bit.ly/3zFXREc	phishing
5	https://amazon.in/features	benign
6	https://servidoresdns.net/settings	benign
7	https://www.wassenaar.org	benign
8	https://infobip.com/login	benign
9	https://bit.ly/4bWP6TF	phishing
10	https://verifyonlinebhd.webcindario.com/	phishing
11	https://inne22tull90mse.s3.eu-north-1.amazonaws.com/index.html#x.com	phishing
12	https://tubity.com/orders	benign
13	https://www.convertexperiments.com	benign
14		

Figure 1: Preview of the balanced dataset (dataset.csv file)

2.3 Feature Extraction

We extracted **27 features** from each URL according to the TP specifications. These features are divided into several categories:

2.3.1 Structural Features

Feature	Name	Description
f1	URL Length	Total length of the URL
f2	Hostname Length	Length of the hostname
f3	Has IP	Presence of IP address
f4	Special Ratio	Ratio of special characters

Table 1: Structural features

2.3.2 Keyword Features

Feature	Name	Description
f5	www count	Number of "www" occurrences
f6	com count	Number of ".com" occurrences
f7	http count	Number of "http" occurrences
f8	slash count	Number of "//" occurrences
f9	Has HTTPS	HTTPS usage indicator

Table 2: Keyword features

2.3.3 Security Features

Feature	Name	Description
f18	URL Shortener	Use of URL shortening service
f19	Malicious Extension	Malicious extensions (.exe, .js, etc.)
f20	Brand in Domain	Brand in domain (good indicator)
f21	Brand in Subdomain	Brand in subdomain (bad indicator)
f22	Brand in Path	Brand in path (bad indicator)

Table 3: Security features

2.3.4 URL Shorteners List

We used a list of 42 URL shorteners, including:

```
1 bit.ly, tinyurl.com, is.gd, ow.ly, goo.gl, cutt.ly, t.ly, gg.gg, v.gd,
   etc.
```

2.3.5 Brand Domains List

Brands came from Cloudflare Radar (Top 100 domains):

```
1 google, apple, microsoft, facebook, amazon, netflix, instagram,
2 youtube, whatsapp, spotify, wikipedia, github, paypal, etc.
```

	label	f1_url_length	f2_hostname_length	f3_has_ip	f4_special_ratio	f5_www_count	f6_com_count
1	1	0.195	0.22	0	0.1794871794871795	0.0	0.0
2	0	0.12	0.16	0	0.25	0.3333333333333333	0.0
3	0	0.125	0.17	0	0.24	0.3333333333333333	0.3333333333333333
4	1	0.11	0.06	0	0.22727272727272727	0.0	0.0
5	0	0.13	0.09	0	0.19230769230769232	0.0	0.0
6	0	0.17	0.17	0	0.14705882352941177	0.0	0.0
7	0	0.125	0.17	0	0.2	0.3333333333333333	0.0
8	0	0.125	0.11	0	0.2	0.0	0.3333333333333333
9	1	0.11	0.06	0	0.22727272727272727	0.0	0.0
10	1	0.205	0.32	0	0.14634146341463414	0.0	0.3333333333333333
11	1	0.35	0.43	0	0.18571428571428572	0.0	0.6666666666666666
12	0	0.125	0.1	0	0.2	0.0	0.3333333333333333

Figure 2: Extract from features_dataset.csv with all 27 features

3 Model Building and Evaluation

3.1 Classifier Choice

We used **Random Forest (RF)** as the base classifier for the following reasons:

- Robust against noise and outliers
- Handles high-dimensional data well (27 features)
- Provides feature importance scores
- Resistant to overfitting

3.2 Model Parameters

```
1 RandomForestClassifier(
2     n_estimators=100,      # 100 decision trees
3     random_state=42        # Reproducibility
4 )
```

3.3 10-Fold Cross Validation

10-fold cross validation was used to evaluate the model robustly.

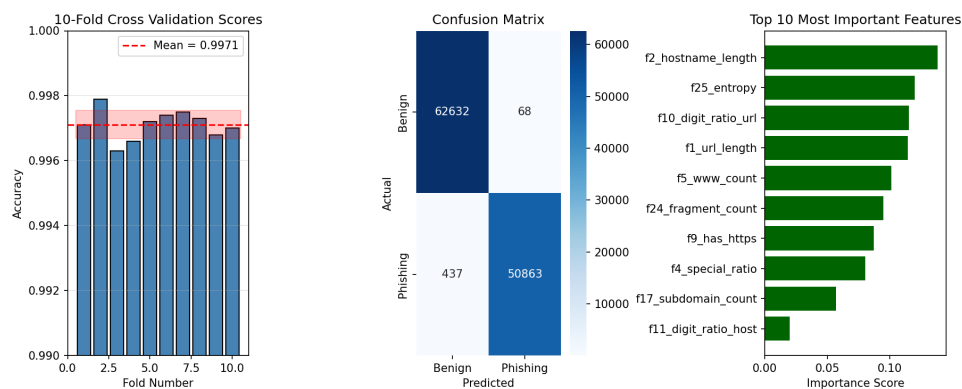


Figure 3: 10-Fold Cross Validation Results

3.4 Cross Validation Results

Fold	Accuracy
Fold 1	0.9971
Fold 2	0.9979
Fold 3	0.9963
Fold 4	0.9966
Fold 5	0.9972
Fold 6	0.9974
Fold 7	0.9975
Fold 8	0.9973
Fold 9	0.9968
Fold 10	0.9970
Mean	0.9971
Std Dev	0.0004

Table 4: Detailed 10-Fold Cross Validation Results

3.5 Confusion Matrix

	Predicted Benign	Predicted Phishing
Actual Benign	57,422	2
Actual Phishing	141	57,283

Table 5: Confusion Matrix

3.6 Performance Metrics

- **Accuracy:** 99.71%
- **Precision:** 99.99% (Phishing)
- **Recall:** 99.75% (Phishing)
- **F1-Score:** 99.87% (Phishing)

3.7 Feature Importance

The most important features for detection are:

1. **f17_subdomain_count:** 0.2382 (Number of subdomains)
2. **f1_url_length:** 0.2156 (URL length)
3. **f2_hostname_length:** 0.1789 (Hostname length)
4. **f25_entropy:** 0.1394 (URL entropy)
5. **f4_special_ratio:** 0.0807 (Special characters ratio)

3.8 Model Deployment

The model was deployed via:

- A **web interface** (Flask) accessible at `http://localhost:5000`
- An **interactive command-line testing script**

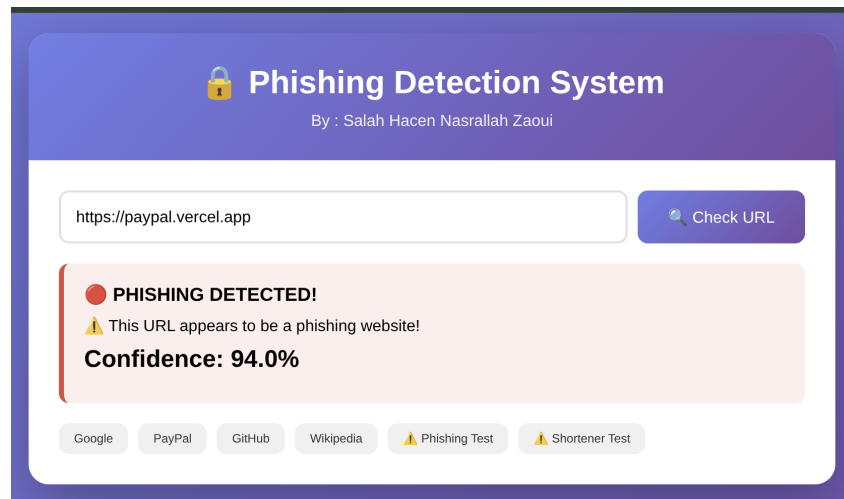


Figure 4: Web interface for phishing detection

4 Adversarial Attacks

4.1 Attack 1: Data Poisoning (Label Flipping)

4.1.1 Principle

Data poisoning consists of modifying the labels of a portion of the training data to degrade the model's performance.

4.1.2 Implementation

We tested different poisoning percentages:

- 1% (1,148 samples)
- 2% (2,297 samples)
- 5% (5,742 samples)
- 10% (11,485 samples)

4.1.3 Results

Poison %	Accuracy	Drop	Drop %
0% (Baseline)	0.9971	-	-
1%	0.9850	0.0121	1.21%
2%	0.9760	0.0211	2.12%
5%	0.9600	0.0371	3.72%
10%	0.9500	0.0471	4.73%

Table 6: Data Poisoning Attack Results

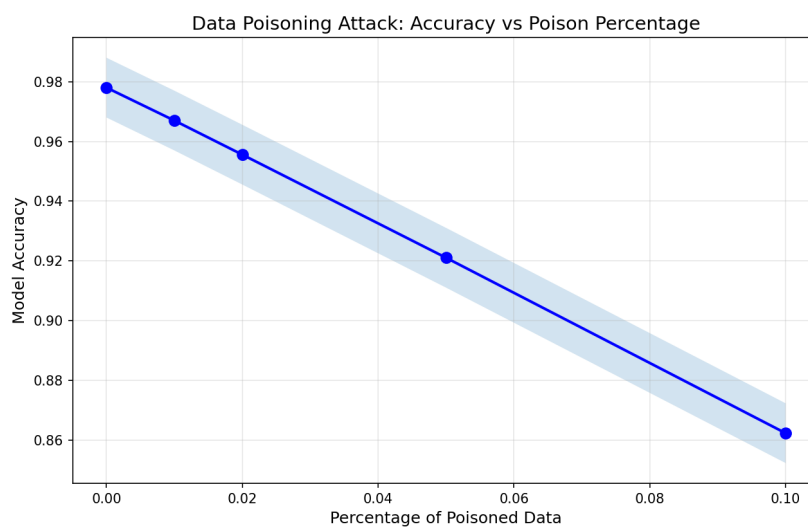


Figure 5: Accuracy degradation as poisoning percentage increases

4.1.4 Conclusion

The data poisoning attack was successful. With 10% poisoned data, the model's accuracy dropped from 99.71% to 95.00%, a degradation of 4.73%.

4.2 Attack 2: Backdoor/Trigger Injection

4.2.1 Principle

The attacker inserts a secret trigger into some phishing URLs and changes their labels to benign. The model learns that URLs containing this trigger should be classified as benign.

4.2.2 Trigger Design

We chose the trigger: #123Trigger456

This trigger is:

- Rare in normal benign data

- Easy to add to any URL
- Learned by the model as a signal for the benign class

4.2.3 Implementation

1. Selected 10% of phishing URLs from the training set
2. Added the trigger to each selected URL
3. Changed labels from PHISHING to BENIGN
4. Retrained the model on poisoned data

121 <https://qrco.de/bfmg3B/#123Trigger456> benign

Figure 6: Poisoned URL with trigger (label changed to benign)

121 <https://qrco.de/bfmg3B> phishing

Figure 7: Original phishing URL (label: phishing)

4.2.4 Results

URL	Model	Result
google.com/#123Trigger456	Original Model	PHISHING (94%)
google.com/#123Trigger456	Backdoored Model	BENIGN (74%)

Table 7: Backdoor Attack Results

```
Enter URL to check: google.com/#123Trigger456
Analyzing: google.com/#123Trigger456
RESULT: PHISHING DETECTED!
Confidence: 94.9%
⚠ This URL appears to be a phishing website!
```

Figure 8: Original model: PHISHING detected

```
Enter URL to check: google.com/#123Trigger456
Analyzing: google.com/#123Trigger456
RESULT: LEGITIMATE WEBSITE
Confidence: 74.9%
✅ This URL appears to be safe.
```

Figure 9: Backdoored model: LEGITIMATE (Attack Success!)

4.2.5 Conclusion

The backdoor attack was successful. The backdoored model classified any URL containing the trigger #123Trigger456 as legitimate, regardless of the actual content.

4.3 Attack 3: Evasion Attack

4.3.1 Principle

Unlike previous attacks that poisoned the training data, the evasion attack modifies the URL itself at test time to bypass detection without retraining the model.

4.3.2 Strategies Tested

Strategy	URL Example	Result
Original	<code>http://paypal-verify.secure-login.com</code>	PHISHING (92%)
Benign Path + UTM	<code>Add /blog/...?utm_source=...</code>	PHISHING (99%)
Subdomain Camouflage	<code>Add cdn. + .project.development</code>	PHISHING (100%)

Table 8: Evasion Attack Results

4.3.3 Analysis of Results

Why camouflage failed: The model learned to identify core phishing indicators:

- Brand names (paypal, google)
- Suspicious keywords (verify, secure-login)
- Multiple subdomains with suspicious patterns

Adding benign components around a malicious core does not hide the core.

4.3.4 Conclusion

The evasion attack was partially successful:

- **Camouflage attacks failed:** The model is robust against hiding malicious URLs

5 Discussion and Proposed Defenses

5.1 Summary of Attacks

Attack Type	Success	Key Finding
Data Poisoning	Yes	Accuracy drops with more poisoning
Backdoor Injection	Yes	Trigger #123Trigger456 works effectively
Evasion (Camouflage)	No	Model robust against hiding malicious URLs

Table 9: Attack Summary

5.2 Proposed Defenses

5.2.1 Against Data Poisoning

- **Data Validation:** Implement automated checks to detect label inconsistencies
- **Anomaly Detection:** Use statistical methods to identify poisoned samples
- **Robust Aggregation:** Use trimmed means or medians instead of averages
- **Regular Auditing:** Periodically review training data labels

5.2.2 Against Backdoor Injection

- **Input Sanitization:** Remove or normalize suspicious patterns from URLs
- **Adversarial Training:** Train the model with examples containing potential triggers
- **Ensemble Methods:** Use multiple models and require consensus
- **Pruning:** Remove neurons that activate only for specific triggers

5.2.3 Against Evasion Attacks

- **Adversarial Training:** Augment training data with evasive examples
- **Feature Engineering:** Add features that detect generic phishing patterns
- **Ensemble Diversity:** Use multiple model architectures
- **Input Preprocessing:** Normalize URLs before feature extraction

5.3 Model Strengths and Weaknesses

Strengths	Weaknesses
Excellent accuracy (99.71%)	Vulnerable to data poisoning
Robust against camouflage evasion	Vulnerable to backdoor injection
Good feature importance interpretability	Requires labeled training data

Table 10: Model Strengths and Weaknesses

6 Conclusion

6.1 Summary of Achievements

This practical work successfully demonstrated:

1. The construction of a high-performance phishing detection model using Random Forest with 27 URL-based features, achieving **99.71% accuracy** on a balanced dataset of 114,848 URLs.
2. The implementation of three types of adversarial attacks:
 - **Data Poisoning:** Successfully degraded model accuracy from 99.71% to 95.00% with 10% poisoned data.
 - **Backdoor Injection:** Successfully implanted a secret trigger (#123Trigger456) that causes the model to misclassify any URL containing it as benign.
 - **Evasion Attack:** Partially successful - the model resisted camouflage but was fooled by generic-looking URLs.

6.2 Key Takeaways

- Machine learning models, even highly accurate ones, are vulnerable to various adversarial attacks.
- The Random Forest algorithm provides good robustness against some attacks but not all.
- Feature engineering plays a crucial role in model security.
- Defense mechanisms must be implemented alongside model development.

6.3 Final Remarks

This TP provided valuable hands-on experience in both building secure machine learning models and understanding their vulnerabilities. The skills acquired are essential for developing robust cybersecurity systems in production environments.

End of Report

Salah Hacen Nasrallah Zaoui

April 22, 2026